

Big data

Module - 1

Big data:

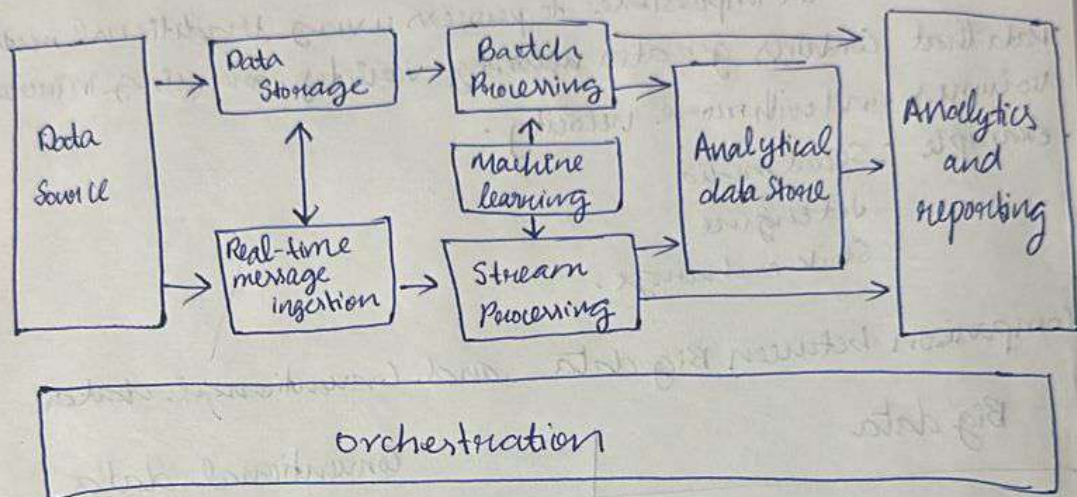
- Big data refers to data that is so large, fast and complex that it's difficult or impossible to process using traditional methods.
- Data that contains greater ~~variety~~ variety, occurring in increasing volumes and with more velocity.
- example : Social media
Jet engine
Stock exchange.

Comparison between Big data and Conventional data

Big data	Conventional data
• Huge data sets • Unstructured data such as text, video and audio • Hard to perform queries and analysis • Needs tools such as Hadoop, HIVE, Hbase, pig and so on • Generated by big financial institutions, Facebook, Google, Amazon etc	• data set size in control. • Normally structured data such as numbers and categories. • Relatively easy to perform queries and analysis. • Tools such as SQL, SAS, R, and excel. • Generated by small enterprises and small banks.

Big data Architecture

A Big data Architecture is designed to handle the ingestion, processing and analysis of data that is too large or complex for traditional database systems.



* Data Source

→ All big data solutions start with one or more data sources.

→ examples:

- Application data stores, such as relational DB.
- Static files produced by applications, such as web server log files.
- Real-time data sources, such as IoT devices.

* Data Storage

→ Data for batch processing operations is typically stored in a distributed file store that can hold high volumes of data in different formats.

→ options for implementing this storage includes Azure data lake store.

Batch processing

It is a method computers use to periodically complete high-volume, repetitive data jobs, such as backups, filtering and sorting.

→ It can be compute intensive and inefficient to run on individual data transactions.

→ Instead, data systems process such tasks in batches, often in off-peak times.

→ example: e-commerce, orders, single (single NO), Full, end-of-the data, as a batch.

* Real-time message ingestion:

→ If the solution include real-time sources, the architecture must include a way to capture and store real-time messages for stream processing.

→ This might be a simple data store, where incoming messages are dropped into a folder for processing.

→ This portion of a streaming architecture is often referred to as stream buffering. options include Azure event hubs, Azure IoT Hub and Kafka.

* Stream processing:

→ After capturing real-time messages, the solution must process them by filtering, aggregation, and preparing the data for analysis.

→ The processed stream data is then written an output sink.

- Azure Stream Analytics provides a managed ~~SQL~~ processing service based on perpetually running SQL queries that operate on unbounded streams.
- You can also use open source Apache Streaming technologies like Spark Streaming.

* Machine learning:

- Reading the processed data for analysis (from batch and stream processing), machine learning algorithms can be used to build models that can predict outcomes or classify data.
- These models can be trained on large datasets, and the resulting models can be used to analyze new data and make predictions.
- This can be done using Azure machine learning.

* Analytical data store:

- Many big data solutions prepare data for analysis and then store the processed data in a structured format that can be queried using analytical tools.
- The Analytical data store is used to store the processed data.

Analytics and reporting :

- Analytics is the technique of examining data and reports to obtain actionable insights that can be used to realise and improve business performance.
- To empower users to analyze the data, the architecture may include a data modelling layer.
- Analysis and reporting can also take the form of interactive data exploration by data scientists or data analysts.

* Orchestration :

- Most big data solutions consist of repeated data processing operations, that transform source data, move data between multiple sources and sinks, load the processed data into an analytical data store.
- Orchestration technology is used to automate the workflow.
- Tools :
 - Azure Datafactor
 - Sqoop
 - Apache Oozie.

Big data platform

- Big data platform is an IT solution which combines several Big data tools and utilities into one package for managing and analyzing Big data.
- There are several open source and commercial Big data platform in the market with varied features which can be used in Big data environment.
- Big data platform generally consist of big data storage, servers, database, big data management, business intelligence and other big data management utilities.
- The primary benefit behind a big data platform is to reduce the complexity of multiple vendors / solutions into a one cohesive solution.

Features:

- Big data platform should be able to accommodate new platforms and tools based on the business requirement.
- It should support linear scale-out.
- It should have capability for rapid deployment.
- It should support variety of data format.
- platform should provide data analysis and supporting tools.
- It should provide real-time data analysis software.

Characteristics of Big data.

- volume
- velocity
- variety
- value
- veracity

Volume:

- It describes the enormous amount of information.
- which is the defining criterion for whether we can consider a dataset as Big data or not.

Velocity:

- velocity describes how rapidly the data is generated and how quickly it moves.
- This data flow comes from sources such as mobile phones, social media, networks, servers etc.

→ For Examples

Variety:

- Variety describes the diversity of the data types and its heterogeneous sources.
- Big data information drawn from a vast quantity of sources, comes in three different types:
 - Structured data
 - unstructured data
 - Semi-structured data.

Veracity:

- veracity describes the data's accuracy and quality.
- Gathered data could have missing pieces, may be inaccurate or may not be able to provide real, valuable insight.
- Veracity, overall, refers to the level of trust there is in the collected data.

Value:

- The most important characteristic from the perspective of the business.
- The value of big data usually comes from insight discovery and pattern recognition that lead to more effective operations, stronger customer relationship etc.
- The more insights derived from the Big data, the higher it's value.

DATA

~~Data is a set of values.~~

⑤
ITA

Meaning of data

Data is a set of value of qualitative and quantitative variables. In other words, pieces of data are individual pieces of information. Data is measured, collected and reported and analysed.

Properties :

- * Flexibility of use
- * Clarity
- * Accuracy
- * Essence
- * Aggregation
- * Compression

Types of DATA

- Structured
- Semi-Structured
- Unstructured

Structured data

Structured data can be defined as the data that resides in fixed field within a record. A certain schema binds it, so all the data has the same set of properties. Structured data is also called relational data. It splits into multiple tables to enhance the integrity of the data by creating a single record to depict an entity.

- example: → spreadsheets
→ Databases such as Oracle, DB2, MySQL
→ online transaction processing systems.

cons:

- Limited flexibility
- Not suitable for all types of data
- Rigid schema
- High maintenance.

Semi-structured data

Semi-structured data is not bound by any rigid schema for data storage and handling. The data is not in the relational format and is not neatly organized into rows and columns like that in a spreadsheet.

It does not follow a strict tabular format like structured data, but still contains tags or markers to separate elements, making it easier to analyze.

- examples: → XML
→ emails
→ JSON
→ Audios.

Unstructured data

Unstructured data is the kind of data that doesn't adhere to any definite schema or set of rules, its arrangement is unplanned and haphazard.

Photos, videos, text documents, and log files can be generally considered unstructured data.

structured data is also known as "dark data" because ^⑥ cannot be analyzed without the proper software tools.

examples: → charts
→ Free-form text
→ web pages
→ videos
→ Audios.

Data Analytic processes and tools.

1. Apache hadoop:

An open-source framework that allows for the distributed processing of large dataset across clusters of computers using simple programming models.

components:

* HDFS: (Hadoop distributed file system): A distributed file system that stores data across multiple machines.

* Mapreduce: A programming model for processing large data sets in parallel.

* YARN: (Yet another resource negotiator): A resource management layer.

2. Apache Spark

A stream processing framework that can process data in real-time. It supports batch processing as well.

Features: → provide high throughput and low latency
→ making it suitable for real-time Analytics

3. Databricks:

A unified analytics platform built on Apache Spark that provides collaborative workspaces for data scientists and engineers.

Features:

- offers tools for data engineering
- machine learning.

4. Teradata:

A data warehousing solution designed for big data Analytics, providing powerful query capabilities on large datasets.

Features:

- supports both on premises and cloud deployments with advanced analytics capabilities.

5. IBM cloud pak for data:

An integrated data and AI platform that helps in organizations to collect, organize, and analyze data across hybrid cloud environments.

Feature: → ^{Business intelligence} Provide tools for data Governance, ML.

Analytics Methods and techniques:

Data Analysis techniques have significantly evolved, providing a comprehensive toolkit for understanding, interpreting and predicting data patterns. These methods are crucial in extracting actionable insights from data, enabling organizations to make informed decisions.

Types:

Descriptive (identify data) Data Analysis:

Descriptive analysis is considered the beginning point for the analytic journey and often serves to answer questions related to what happened. This technique follows ordering factors, manipulating, and interpreting varied data from diverse sources and then turning it into valuable insights.

example: Sales performance: A retail company might use descriptive statistics to understand the average sales volume per store or to find out which products are the best sellers.

Qualitative Data Analysis

Qualitative data analysis techniques cannot be measured directly, and hence, this technique is utilized when an organization needs to make decision based on subjective interpretation. For instance, qualitative data can

Can involve evaluating customer feedback, impact of survey questions, the effectiveness of social media posts etc.

Predictive Data Analysis

Predictive data analysis allows us to forecast future events by addressing enquiries regarding potential outcomes. individuals need to utilize the results of descriptive data analysis, exploratory and diagnostic analysis techniques and combine machine learning and artificial intelligence.

example:

credit scoring: financial institutions use predictive models to assess a customer's likelihood of defaulting on a loan.

Diagnostic data Analysis

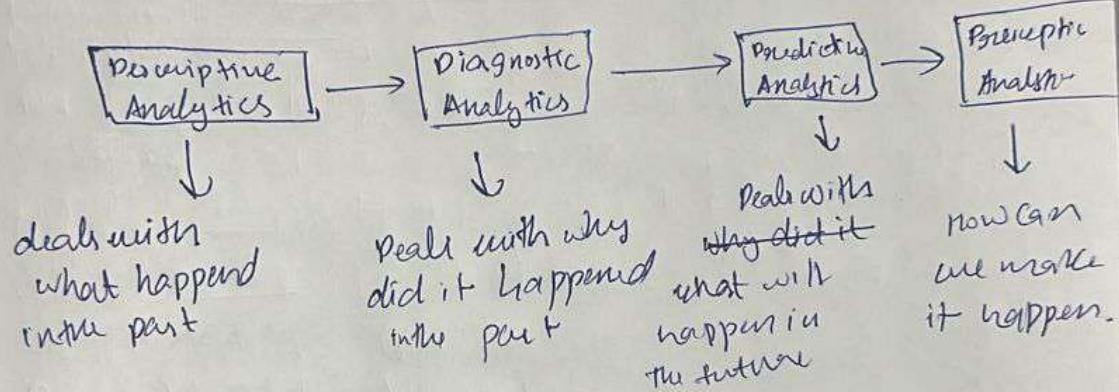
understanding the reasons behind an occurrence simplifies the process of determining the "How" related to that particular aspect. For instance, with diagnostic analysis, we can identify why our sales results are declining and eventually explore that exact factors that led to their loss.

example: inventory Analysis: checking if lower sales correlate with stock out or overstock situations.

Intelligent Data Analysis

(8)

IDA or intelligent analysis, is a data analysis techniques that relies on artificial intelligence, machine learning, high performance computers, pattern recognition, statistics and database and visualisation technology. IDA is a tool that use advanced techniques to analyze vast quantities of data in order to get valuable insights and information.



IDA consist of three distinct stages → NoKKonam.

Bigdata Analytics life cycle : NoKKonam.

Module-2

Stream:

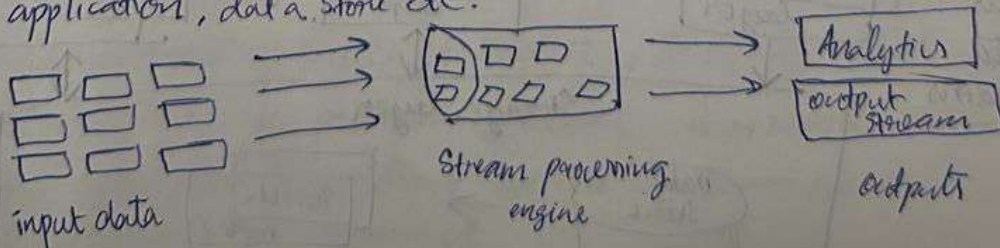
- Stream Refers to a sequence of data elements or symbols made available over time.
- Data stream transmits from a source and receives at the Processing end in a network.
- A continuous stream of data flows between the source and receiver ends, and which is processed in real-time.

Features:

- Stream Computing is a way to analyze and process Big data.
- High rate of receiving data
- Processing is done in real time to gain current insights to take ~~app~~ appropriate decisions.
- Implements in a distributed clustered environment.

* Stream processing is a method of data management that involves continuously receiving and promptly analyzing, filtering, transforming or enhancing a continuous data stream in real time.

* After undergoing processing, the data is then directed towards an application, data store etc.



Reasons: nokkanam.

Stream Computing domains:

- Financial Sectors
- Business intelligence
- Risk management
- Search engines.

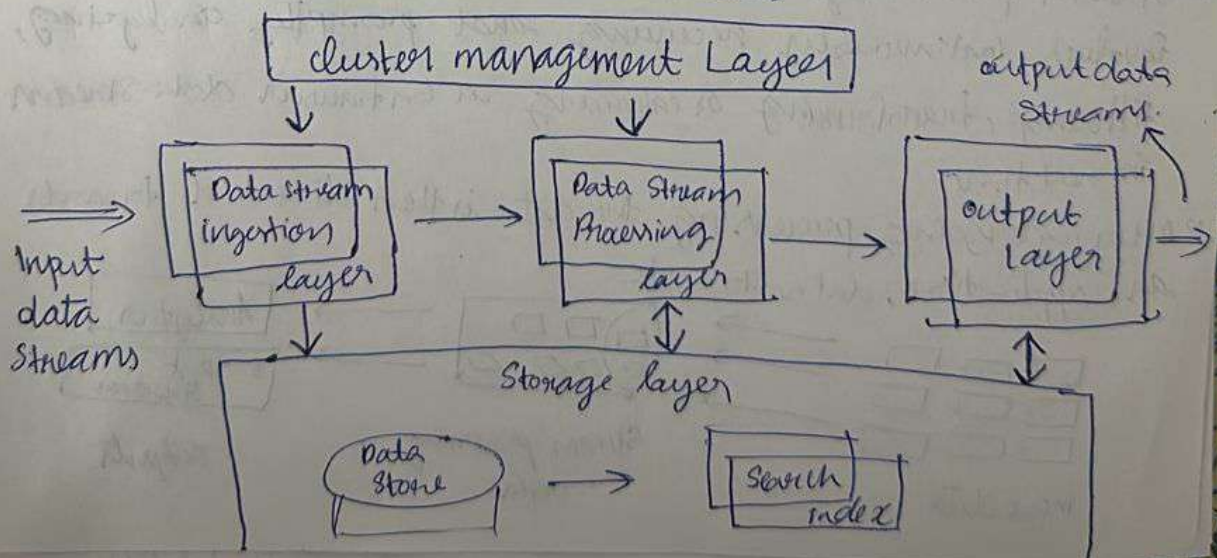
Popular Stream processing frameworks:

- Apache Spark
- Apache Storm
- Apache Samza
- Apache Flink.

Issues:

- Stream often deliver elements very rapidly
- Efficient algorithm is needed
- Important to store the data
- Data can be complex types - Document, Images, time series.

Streaming data architecture



Input data stream: There are Continuous flows of data that come from various sources such as sensors, applications, devices etc. (2)

2. Data Stream ingestion layer: To collect and route the incoming data streams. Think of it like a gate that manages which data enters the system and how it's organized.

3. Data stream processing layer: To analyze and transform the ingested data in real-time. This could be filtering, aggregation, joining and enriching the data. It's the brain of the system that performs computation on live data.

Output layer: To forward the processed data to external systems, dashboards, storage or users. This is where insights and results from stream knowings are sent out.

5. Storage layer: Data Store: persists raw or partially processed data for future use.

• Search index: Allows fast searching and querying of stored data.

• Cluster management layer: oversees and coordinates the resources, fault tolerance, scaling and distribution of processing tasks. It ensures smooth and efficient execution of the system in a distributed environment.

Data Stream model

- The data stream model is a computational model used to analyze and process continuous streams of data.
- With stream model data is considered to be infinite and unbounded, and algorithms are designed to make decisions and real-time computation.
- Commonly used in various applications such as
 - monitoring systems
 - network traffic analysis
 - sensor data processing
- There are different ways of modeling data stream, querying, processing and management.

Graph-based stream data model

- It involves representing the data stream and its transformation using a graph structure.
- In this model, each node in the graph represents an operator, while each edge represents the flow of data.
- Each node in the graph performs specific ~~task~~ data transformation or computations on incoming data streams.
- These transformations may include filtering, aggregation, enrichment or joining of data streams.

Relation-oriented stream data model

- This model is specifically designed for representing and processing streaming data as tuple organized into relations, similar to relational databases.
- In this model, streaming data is represented as a sequence of tuples, where each tuple corresponds to a single data event.

in unbounded and time-ordered sequence of data items^③ in the data stream model.

→ Each tuple consists of a set of attributes, like a row in a database table.

Sampling

- Process of collecting a representative collection of elements from entire stream.
- usually much smaller than the entire stream data
- Retains all the significant characteristics and behaviour of the stream.
- used to estimate / predict many crucial aggregates on the stream.

Techniques:

1. Fixed proportion Sampling.

- Samples fixed proportion of data
- used when you are aware of the length of data
- Ensures representative sample.
- useful for large volumes
- Less biased than fixed sized sampling
- May lead under/over representation.

example:

we have a Big data set with 10 million user records from a retail app. if you want to apply fixed sampling:

You decide to sample 1% of the data → so you extract 100,000 records

2. Fixed Size Sampling:

- Samples fixed number of data points.
- Does not guarantee representative sample.
- useful for reducing data volume.
- Can be biased if data is not ~~evenly~~ randomly distributed.
- Less effective when data size increases.

Example:

You have a dataset with 5 million transaction records, and you want to analyze just 1,000 for testing purpose.

You apply fixed size sampling to select exactly 1000 records.

3. Concise Sampling

- Goal is to maintain a small reservoir of a fixed size while still achieving representative sampling of the data stream.
- Number of samples that can be stored at a given time is limited, which can be a challenge while dealing with large data streams.
- Size of the sample may need to be adjusted based on the amount of memory available to store the data.
- Instead of choosing random samples, the sampling algorithm may prioritize choosing samples with unique or representative values.

Example: • YouTubers
• No. of views

Bloom - filtering

(4)

Space-efficient data structure

- Used to check whether an element belongs to a set.
- Probably says that the element belongs to a set (False positive).
- Accurately says that the element does not belong to a set (Only True negatives).
- Hence, Recall rate is 100%

example:

Insert elements 10 and 7 in the bloom filter of size 5.
Consider these two hash functions:

$$h_1(x) = x \bmod 5$$

$$h_2(x) = (2x + 6) \bmod 5$$

Comment on the presence of elements 10 and 15.

Ans: • m (Size of Bloom filter) = 5

• hash functions;

1. $h_1(x) = x \bmod 5$

2. $h_2(x) = (2x + 6) \bmod 5$

• Insert elements: 10
7

element to insert	$h_1(x)$	$h_2(x)$	Bloom filter										
10	$10 \bmod 5$ 0	$(2(10)+6) \bmod 5$ 1	<table><tr><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	1	1	0	0	0	0	1	2	3	4
1	1	0	0	0									
0	1	2	3	4									
7	$7 \bmod 5$ 2	$(2(7)+6) \bmod 5$ $20 \bmod 5$ 0	<table><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	1	1	1	0	0	0	1	2	3	4
1	1	1	0	0									
0	1	2	3	4									

checking for presence

element	$h_1(x)$	$h_2(x)$	Presence
14	$14 \bmod 5$ 4	$(2(14)+6) \bmod 5$ 4	At index 4, the value is 0 which means that the element is not present in the set (accurately says absence). • The value is 1 at both the index 0 and 1 which depicts the element is present in the set. But in reality it is not. (False positive)
15	$15 \bmod 5$ 0	$(2(15)+6) \bmod 5$ $36 \bmod 5$ 1	

Applications:

- Bloom filter can be used on first layer i.e. filtering.
- Facebook uses bloom filter to avoid one-hit wonders.
- medium uses Bloom filter in its Recommendation module to avoid showing the post that the user has already seen.

Flajolet - Martin algorithm

Purpose: This algorithm is used to count how many unique (distinct) elements are in a large data stream, without storing all of them.

- It's great for huge datasets, where storing all items is not possible.
- useful for:
 - counting how many different words are on a webpage.
 - Finding how many different pages a user has visited
 - tracking unique logins.

(5)

Probabilistic Counting: Instead of counting directly, it uses random methods to estimate the number of unique items.

Hash functions: Convert each item into a string of 0's and 1's (called a hash) using a special function. This helps spread items out randomly.

Steps of Algorithm

Step 1: Pick a hash function:

This function turns each item into a binary number (eg: 01001).

Step 2: Count trailing zeros:

For each hashed value, count how many zeros are at the end of the binary number.

Step 3: Find the maximum trailing zeros:

Pick the largest number of trailing zeros from all the items.

Step 4: Estimate the unique count using the formula:

$$\text{Number of distinct elements} = 2^R$$

$R \rightarrow$ max trailing zeros.

example:

Data stream : 4, 2, 5, 9, 1, 6, 3, 7

hash function:

$$h(x) = (x + 6) \bmod 32$$

Hashed values :

$$h(4) = 10 \rightarrow 01010 \rightarrow 1 \text{ trailing zero}$$

$$h(2) = 8 \rightarrow 01000 \rightarrow 3 \text{ trailing zero}$$

$$h(5) = 11 \rightarrow 01011 \rightarrow 0$$

$$\vdots$$
$$h(7) = 13 \rightarrow 01101 \rightarrow 0$$

Trailing zeros for all items:

$$\{1, 3, 0, 0, 0, 2, 0, 0\}$$

maximum trailing zero's (R) = 3

$$\text{So estimated unique elements} = 2^3 = \underline{\underline{8}}$$

DCIM

Data - Grigori - Indyk - Motwani

- Commonly called as Motwani Algorithm or DCIM
- used to find the number of 1's in a stream of data.
- uses $O(\log^2 N)$ bits to represent a window of N bits
- Error rate is no more than 50%

(5)

ments:

Timestamp:

- each element entering in the stream will be allotted a timestamp based on the position of it.
- example: If first bit has timestamp 1, then second bit will have timestamp 2, third bit 3 and so on...

2. Buckets:

- used to represent time intervals in a data stream
- Algorithm divides the stream into buckets, each will have size of power of 2.
- Bucket contains the bits 0 and 1

Rules for forming a Bucket

1. Every bucket should contain at least a single 1 in it
2. Right side of the bucket should strictly start from 1
3. Length of the bucket is equal to the number of 1's in it.
4. Every bucket length should be in power of 2.
5. As we move to left, the bucket size should not decrease.
6. No more than 2 buckets can have same size

Example:

• consider the following stream:

Stream = 4 10101111 001 000 1100101...

N = 20

4 = 2^2

2 = 2^1

1 = 2^0

Form buckets:

4 = 2^2 1 0 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 1 0 1

- ③ IF new entering bit is 0, no change
 IF new entering bit is 1, alter the buckets

Seminode choikanam:

- Bucket size of power of 2.
- Bucket contains the bits 0 and 1.
- Bucket for forming a bucket.
- Every bucket should contain at least a single unit.
- Right side of the bucket should be empty.
- Length of the bucket is equal to the number of 1's in it.
- Every bucket length should be in power of 2.
- When move to left, the bucket size should not decrease.
- The time taken to bucket is $O(n \log n)$.

Example:

Consider the following stream:
 Stream = "... 1010111 001 000 1100111..."
 Buckets = 1 0 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 1 1 1

Module-3

①

RDBMS

- Relational Database Management System
- Suitable for structured data
- OLTP
- when the data needs consistent relationship
- Needs expensive hardware or high-end processors to store huge volumes of data

Hadoop

- Hadoop is an open source framework that is meant for storage and processing of big data in a distributed manner.

Hadoop

- Node based flat structure.
- Suitable for structured, unstructured data, supports variety of formats (xml, json).
- Analytical, big data processing. Hadoop clusters, node acquire any consistent relationships between data.
- Big data processing, which does not require any consistent relationship between data.
- In commodity hardware less configure hardware.

- it is the best solution for handling big data challenges.

Features:

- ① Open Source: Hadoop is an open source framework which means it is available free of cost.
- ② Distributed processing: Hadoop is supported by distributed processing of data i.e. faster processing. The data in Hadoop HDFS is stored in a distributed manner.
- ③ Fault tolerance: Hadoop is highly fault-tolerant. It creates three replicas for each block.
- ④ Reliability: Hadoop stores data on the cluster in a reliable manner that is independent of machine.
- ⑤ Scalability: It is compatible with the other hardware and we can easily add / remove the new hardware to the nodes.
- ⑥ High availability: The data stored in Hadoop is available to access even after the hardware failure.

Core Components:

1. HDFS (Hadoop distributed file system) - HDFS is the basic storage system of Hadoop. The large data files running on a cluster of commodity hardware are stored in HDFS.

Key aspects of HDFS are:

- Storage component
- Distributes data across several nodes.
- Node is redundant.

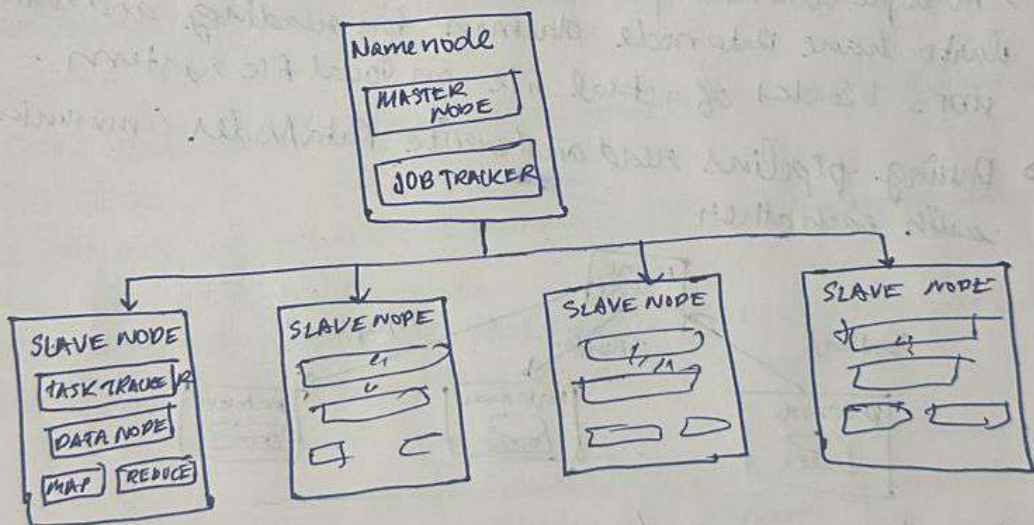
map reduce: Map reduce is the hadoop layer that is responsible for data processing. It is responsible for Parallel processing of high volume of data by dividing data into independent tasks.

The map is the first phase of processing that specifies complex logic code and the reduce is the second phase of processing that specifies light weight operations.

Key aspects of map reduce:

- Computational framework
- Splits a task across multiple nodes
- processes data in parallel.

Hadoop architecture



Hadoop architecture is a distributed Master-slave architecture.

Master HDFS: Its main responsibility is partitioning the data storage across slave nodes. It also keeps track of locations of data on slave nodes.

Master Map reduce: It decides and schedules tasks on slave nodes.

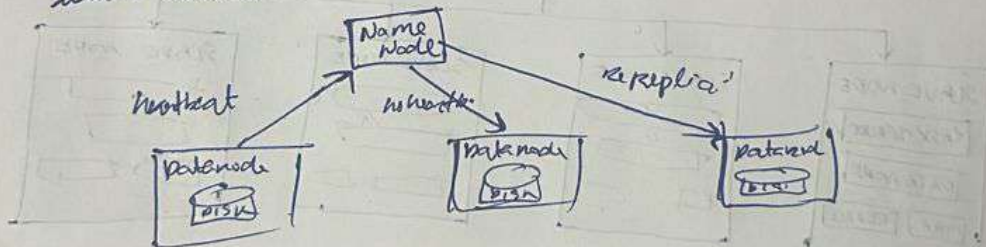
HDFS Daemons:

(i) Name node:

- The namenode is the master of HDFS that directs the slave Datanodes to perform I/O tasks.
- Blocks: HDFS breaks large file into smaller pieces called blocks.
- RackID: NameNode uses rackID to identify data nodes in the rack.

(ii) Data node:

- Multiple data nodes per cluster. Each slave machine in the cluster has DataNode daemon for reading and writing HDFS blocks of actual file on local file system.
- During pipeline read and write DataNodes communicate with each other.



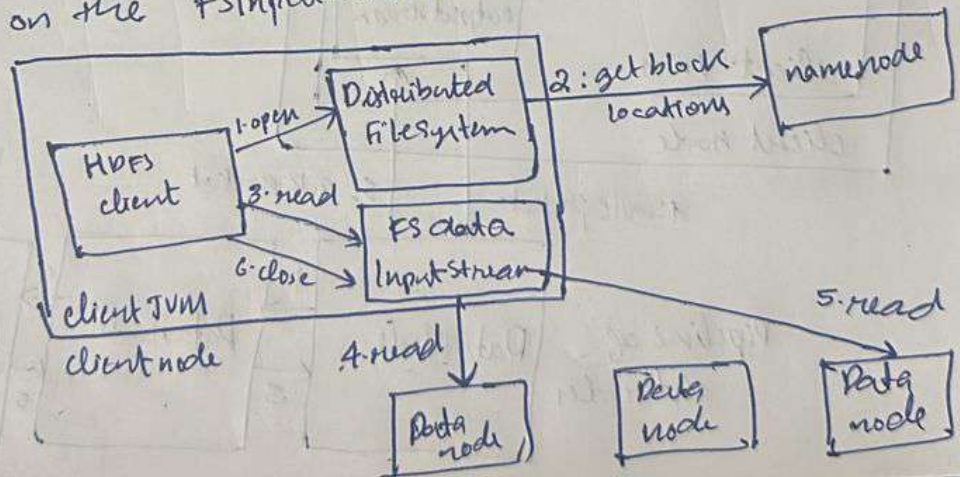
(iii) Secondary name node

- Takes Snapshot of HDFS metadata at intervals specified in the hadoop Configuration.
- Memory is same for Secondary node as name node.
- But Secondary node will run on a different machine.

- features of HDFS :
- Storage Component of Hadoop
 - Highly fault tolerant
 - Distributed file system
 - Inspired by Google file system
 - Replication
 - provides high throughput access.

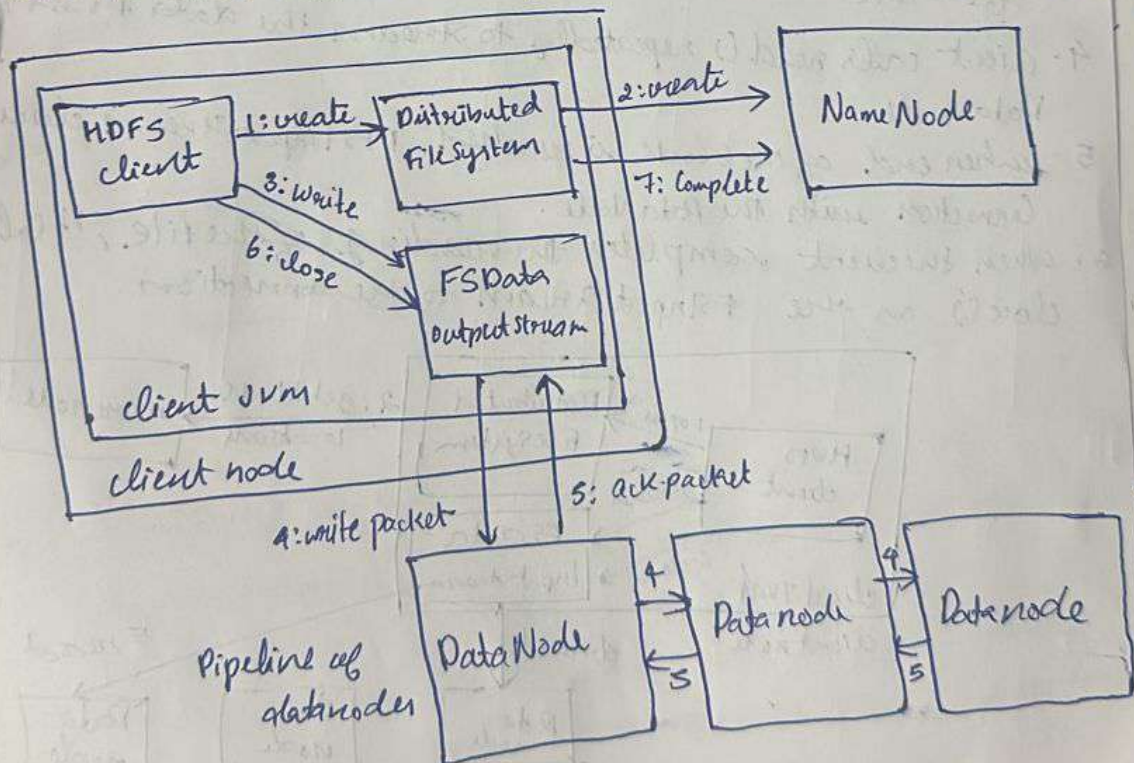
File Read operation

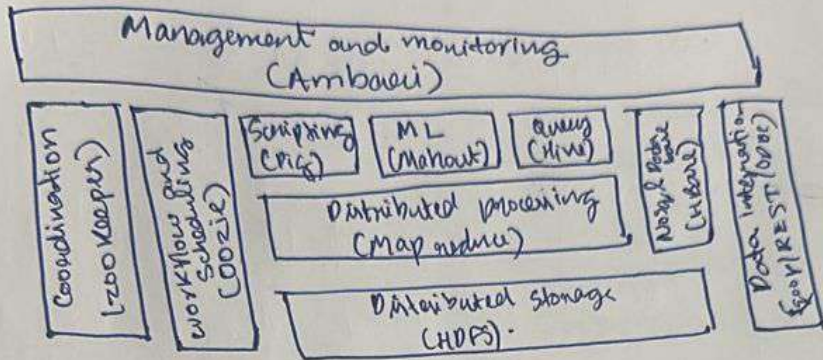
1. The client opens the file that it wishes to read from by calling `open()` on the DFS.
2. The DFS communicates with the Namenode to get the location of data blocks. Namenode returns with the address of the Data nodes that ^{the data} blocks are stored in.
3. client then calls `read()` on the stream `DFSInputStream`, which has addresses of DataNodes for the first few blocks of the file.
4. client calls `read()` repeatedly to stream the data from the Data node.
5. when end of the block is reached, `DFSInputStream` closes the connection with the DataNode.
6. when the client completes the reading of the file, it calls `close()` on the `FSInput Stream` to the connection.



File Write operation.

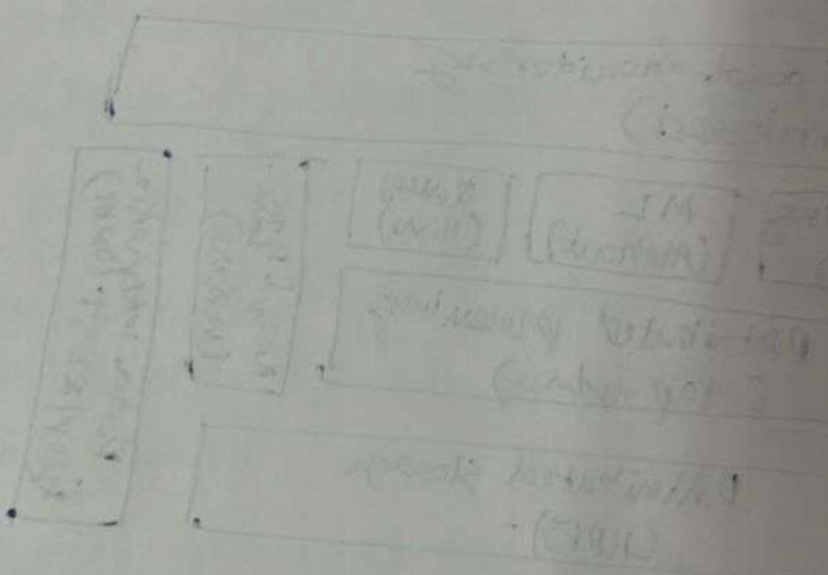
1. The client calls `create()` on DistributedFileSystem to create.
2. An RPC call to the namenode happens through the DFS to create a new file.
3. As the client writes data, data is split into packets by DFS OutputStream, which in turn writes to an internal queue called data queue.
4. Data streamer streams the packets to the first DataNode in the pipeline. it stores packet and forwards it to the second DataNode in the pipeline.
5. In addition to the internal queue, ~~DFS~~ DFS OutputStream also manages an "AckQueue" of the packets that are waiting for acknowledgment by Data nodes.
6. When the client finishes writing the file, it calls `close()` on the stream.





1. HDFS: Hadoop distributed File system. it simply stores data files as close to the original form as possible.
2. HBase: It is Hadoop's distributed column based database. it supports structured data storage for large tables.
3. Hive: It's a Hadoop's data warehouse, enables analysis of large data sets using a language very similar to SQL.
4. Pig: Pig is an easy to understand data flow language. It helps in the analysis of large data sets.
5. ZooKeeper: It is an open source application that configures and synchronizes the distributed systems.
6. Oozie: It is a workflow scheduler system to manage Apache Hadoop jobs.
7. Mahout: It is a scalable machine learning and data mining library.
8. Chukwa: It is a data collection system for managing large distributed systems.
9. Sqoop: it is used to transfer bulk data between Hadoop and structured data stores such as relational DB.
10. Ambari: it is a web based tool for provisioning, managing and monitoring Apache Hadoop cluster.

Mapreduce } no Kknam.



MapReduce is a distributed system for processing large volumes of data. It is designed to handle data that is too large to fit on a single machine. The system is divided into two main parts: the Map phase and the Reduce phase. In the Map phase, the data is split into small chunks, and each chunk is processed by a MapTask. The results of the MapTasks are then sent to the Reduce phase, where they are combined and processed by a ReduceTask. The system is designed to be scalable, meaning that it can handle large amounts of data by adding more machines to the cluster. It is also designed to be fault-tolerant, meaning that it can continue to operate even if some of the machines in the cluster fail.

Module-4 Big data

①

Pig

- Pig is a high level data flow platform for analyzing large datasets stored in hadoop clusters.
- Pig simplifies the process of writing complex map-reduce programs.
- In a Mapreduce framework, programs need to be translated into series of map and reduce stages.
- To bridge this gap an abstraction called pig was built on top of hadoop.
- Apache pig reduces the time of development using the multi-query approach.
- Pig is beneficial for programmers who are not from java background.

Features:

- * Ease of programming: Writing complex java programs for map reduce is quite difficult for non-programmers, pig makes the process easy.
- * Optimization opportunities: By encoding tasks in a certain way, the system is able to automatically optimize their execution.
- * Extensibility: A user defined function is written in which the user can write their logic to execute over the dataset.
- * Flexible: It can easily handle structured as well as unstructured data.
- * In-built operators: It contains various type of operators such as sort, filter and join.

Advantages

- Less code
- Reliability
- Nested data types.

Apache pig execution mechanism: nokkanna

Apache pig execution modes

* Local mode

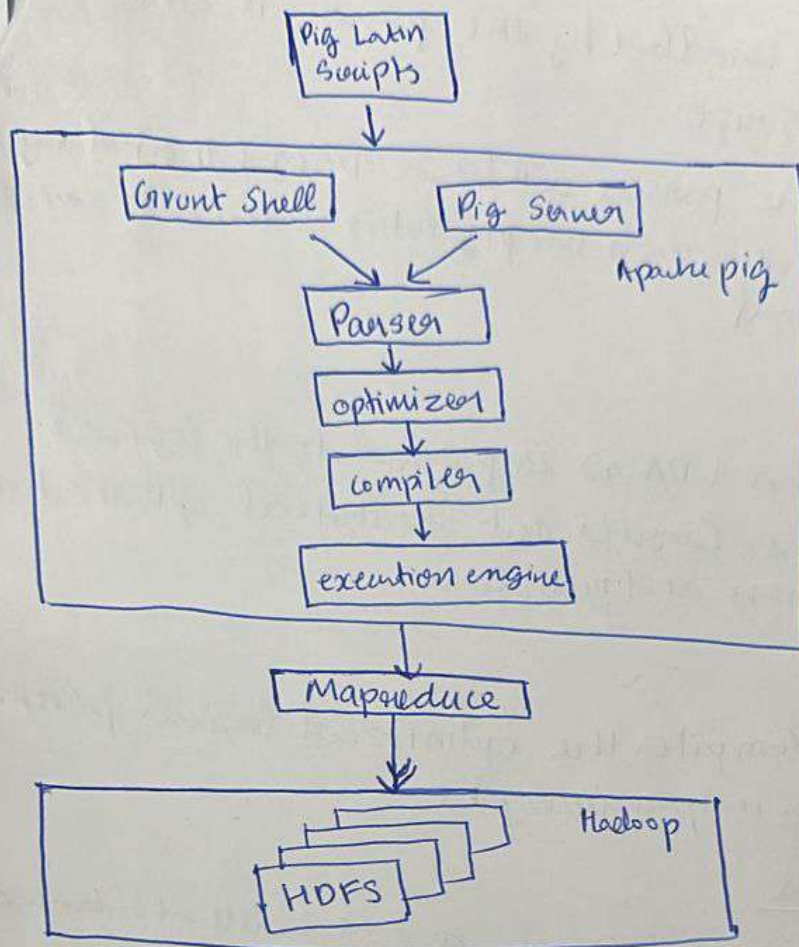
- In this mode, all the files are installed and run from your local host and local file system.
- There is no need of Hadoop and HDFS.
- This mode is generally used for testing purpose.

* Mapreduce Mode

- Mapreduce mode is where we load on process the data that exists in the Hadoop file system (HDFS) using Apache Pig.
- It is known as Hadoop mode.
- In this mode, whenever we execute the pig latin statements to process the data, a mapreduce job is invoked in the back-end to perform a particular operation on the data that exists in the HDFS.

Pig	SQL
• Pig latin is a procedural language • Schema is optional • The data model in Apache Pig is nested relational • limited opportunity for query optimization	• SQL is a declarative language. • Schema is mandatory in SQL • The data model used in SQL is flat relational. • more opportunity for query optimization.

Architecture



Pig latin :

- * The language used to analyze data in hadoop using pig is known as pig latin
- * it is a high level data processing language which provides a rich set of data types and operators to perform various operation on the data.

Grunt shell

- interactive shell for running pig commands.

Parser

- * Pig scripts are handled by the parser, it checks the syntax of the script.
- * The output of the parser will be a DAG (directed acyclic graph), which represents the pig latin statements and logical operators.

Optimizer

- * The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations such as projection and pushdown.

Compiler

- * The compiler compiles the optimized logical plan into a series of mapreduce jobs.

Execution engine

- * Finally the mapreduce jobs are submitted to Hadoop in a sorted order.
- * These mapreduce jobs are executed on Hadoop producing desired results.

GRUNT

- * Grunt is pig's interactive shell. it enables users to enter Pig latin interactively and provides a shell for users to interact with HDFS.
- * Grunt provides command-line history and editing.
- * You can enter pig latin directly into Grunt.

Latin

structure

(3)

- pig latin program consists of a collection of statements.
- A statement can be thought of as an operation or a command.
- Statements are usually terminated with a semicolon.
- Grant do not need the terminating semicolon.

Pig latin Commands

Cat → prints the contents of one or more files.

Cd → changes the current directory

Cp → copies a file or directory to another directory

ls → lists files

Mkdir → creates a new directory

Mv → moves a file or directory to another directory

Rm → Deletes a file or directory.

LOAD AND STORE

Load function:

A function that specifies how to load data into a relation from external storage.

Store function:

A function that specifies how to save the contents of a relation to external storage.

Built in functions (pig)

- AVG → calculates the average value of entries in a bag.
- CONCAT → Concatenates byte arrays or character arrays together.
- COUNT → calculates the number of non-null entries in a bag.
- MAX → calculates the maximum value of entries in a bag.
- MIN → calculates the minimum value of entries of a bag.
- SUM → calculates the sum of values of entries in a bag.
- SIZE → calculates the size of a type.
- DIFF → calculates the set difference of 2 bags.

HIVE

- Hive is a data warehouse system which is used to analyze structured data.
- It is built on top of Hadoop and developed by Facebook.
- Hive provides the functionality of reading, writing, and managing large datasets.
- It runs SQL-like queries called HQL (Hive Query Language).
- Hive supports DDL, DML, and user defined functions.

Features

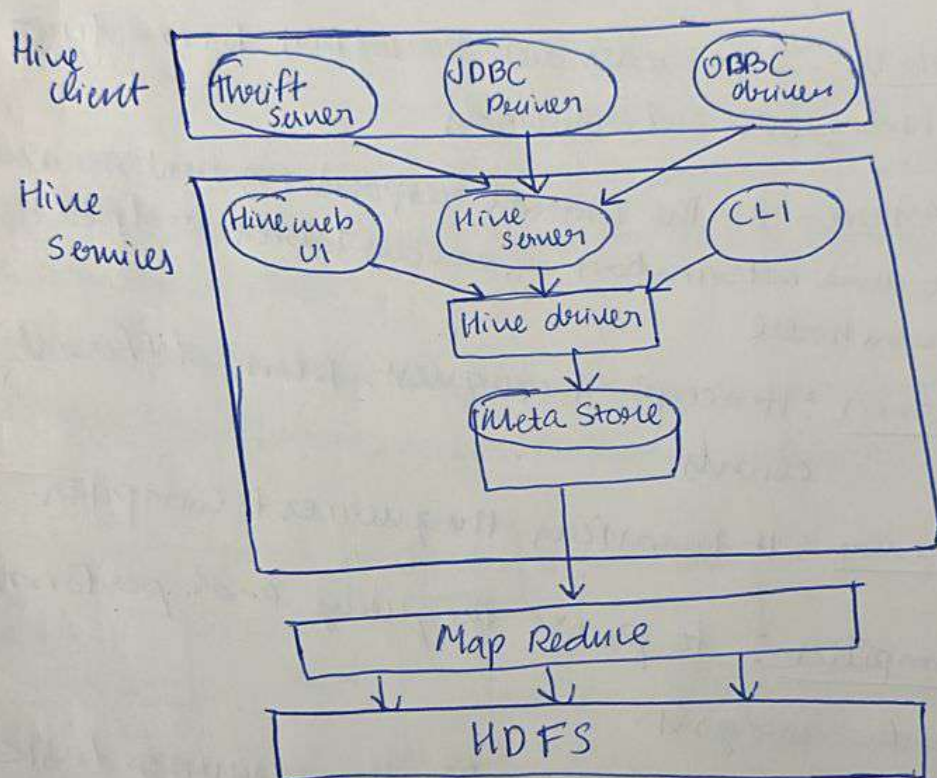
- Hive is fast and Scalable.
- It provides SQL-like queries that are implicitly transformed to mapreduce or spark jobs.

Capable of analyzing large datasets stored in HDFS. (7)

It uses indexing to accelerate queries.

It can operate on compressed data stored in the Hadoop.

Hive Architecture



Hive client: A component that enables clients to submit queries to hive and retrieve results over a network connection.

Thrift server $\Rightarrow$ it is a cross-language service provider platform that serves the requests from all those programming languages.

JDBC driver $\Rightarrow$ It is used to establish a connection between hive and Java applications.

- ODBC driver : It allows the applications that support the ODBC protocol to connect to Hive.

• Hive Services

Limitation in hadoop: no

- Hive CLI - The Hive CLI is a shell where we can execute hive queries and commands.
- Hive web UI - It provides web-based GUI for executing hive queries and commands.
- Hive metastore - It is the central repository that stores all the structure information of various tables and partitions in the warehouse.
- Hive Server : It accepts the requests from different clients.
- Hive driver : It transfers the queries to compiler.
- Hive compiler : to parse the query and perform semantic analysis.
- Hive execution engine : executes the incoming tasks.

RDBMS	HIVE
• it is used to maintain database • It uses SQL • Schema is fixed • Normalized data is stored • It doesn't support Partitioning	• it is used to maintain data warehouse • it uses HQL • Schema varies in it • Normalized and denormalized Both type of data is stored. • It supports automatic Partition.

HBASE

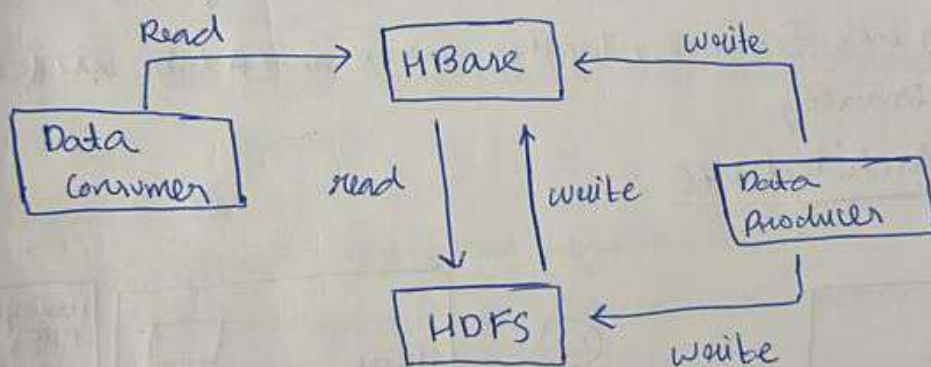
(5)

HBase is a distributed column-oriented database built on top of the Hadoop file system.

→ Stores and retrieves data in a column-wise, the data in a table is stored and accessed column by column rather than row by row.

→ The values of each column are stored together in contiguous memory locations.

- HBase is a data model similar to Google's BigTable designed to provide quick random access to huge amount of structured data.
- It is a part of Hadoop ecosystem that provides random real-time read/write access to data in Hadoop file system.



HDFS

- HDFS is a distributed file system suitable for storing large files.
- HDFS does not support fast individual record lookups.
- It provides only sequential access of data.

HBASE

- HBase is a database built on top of the HDFS.
- HBase provides fast lookups for large tables.
- HBase internally uses hash tables and provides random access.

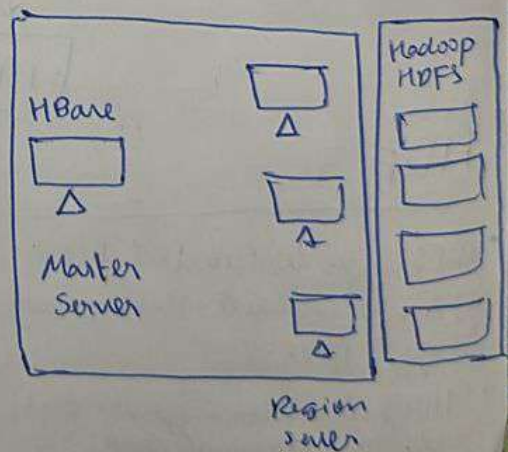
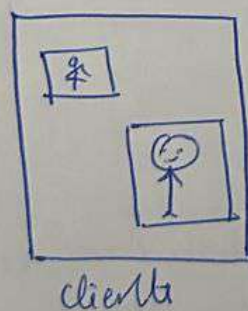
Features

- HBase is ^alinearly Scalable
- It has automatic failure support
- It provides consistent read and write
- It has easy java API for client.
- It provides data replication across clusters.

Where to use / Application

- Apache HBase is used to have random, real-time read/write access to Big data.
- It hosts very large tables on top of clusters of commodity hardware.
- Apache HBase is a non-relational database modeled after Google's Bigtable.
- it is used whenever there is a need to write heavy applications
- Companies like Facebook, Twitter, Yahoo and Adobe use HBase internally.

Hbase - Architecture



It has three major components:

- client library
- master server
- Region server.

Master Server

- Master server assigns regions to the region servers and takes the help of Apache Zookeeper for this task.
- It handles load balancing
- it is responsible for schema changes
- it unloads the busy servers and shifts the regions to less occupied servers
- It maintains the state of the cluster by negotiating the load balancing.

Regions and Region servers

- Regions are tables that are split up and spread across the region servers.
- Region server:
 - The region server has regions that -
 - Communicate with the client and handles data-oriented operations
 - Handle read and write requests for all regions under it.

Zookeeper: not known.

①

①

- ①

①

①

①

①

①

file.choose(): In R it's also possible to choose a file using the function file.choose(), and if you're a beginner programming then this method is really useful for you.

```
myfile = read.delim (file.choose(), header = FALSE)
```

```
Print(myfile).
```

Reading CSVs

The best way to read data from a CSV file is to use read.table().

We can also use read.csv(), which is a wrapper around read.table with sep argument preset to a comma (,). The result of using read.table is a data frame. The first argument to read.table is the full path of the file to be loaded.

```
tomato <- read.table (file = "tomato.csv",  
                      header = TRUE, sep = ",")  
head(tomato).
```

Data cleaning

The data collected might contain insufficient data or there might be duplicate records of the data. data cleaning doesn't mean changing the meaning of data. it means identifying problems caused by data collection, processing and storage process. data cleaning is the process of correcting or deleting inaccurate, damaged, improperly formatted, duplicated and insufficient data from a dataset.

Steps for data cleaning.

1. Remove duplicate and irrelevant observation:

Remove duplicate or pointless observations as well as undesirable observations from your dataset. The majority of duplicate observation is occurred during data gathering. Duplicate data can be also produced when you merge data sets of several source.

2. Fix structural errors.

When you measure or transfer data and find odd naming practices or wrong capitalization, such are structural faults. Mis-labelled categories or classes may result from these inconsistencies.

3. Filter unwanted outliers.

There will frequently be isolated findings that, at first glance, do not seem to fit the data you are analyzing. Removing an outlier if you have a good reason to, such as incorrect data entry, will improve the performance of the data you are working with.

Matrices

* A matrix is a two dimensional data set with columns, rows.

* A column is a vertical representation of data, where row is a horizontal representation of data.

* In R, a matrix is created with the `matrix()` function to get no of rows and columns, specify n row and m column.

eg: matrix 1 ← matrix (C(1,2,3,4,5,6), nrow=3,

> # print the matrix

> matrix 1

- * To access the items, you can use [] brackets. The first number in the bracket specifies the row position and the second number specifies the column position.

eg: matrix 2 ← matrix (C("Apple", "banana", "cherry", "orange"),
nrow=2, ncol=2)

> matrix [1,2].

- * To access the whole row, specify a comma after the number of rows in the brackets

> this matrix [2,]

- * To access the whole column, specify just comma before the number of columns

> this matrix [,2]

- * To find the transpose of a matrix we use t [] function

eg: we have a matrix B

transpose of B ⇒ t[B]

- * To add a column to the matrix we can use cbind () function.

eg: newmatrix ← cbind (matrix 4, C("straw", "cherry", "blue"))

- * To add a row to the matrix, we can use rbind () function

function used in descriptive statistics

(1)

(3)

- * mean(x): Calculate the arithmetic mean of a vector
- * median(x): Returns the median (middle value) of x.
- * mode(x): In base R, there's no direct mode function for the most frequent value. But mode(x) gives the datatype, not the statistical mode.
- * sd(x): Computes the standard deviation of x.
- * summary(x): gives the quick statistical summary.
- * range(x): gives the maximum and minimum value of x.

Applications of R programming.

1. Statistical Analysis
2. Data visualization
- * 3. Data Science and machine learning
- * 4. Finance and economics.
5. Reporting and documentation
6. Web application
7. Bioinformatics and genomics.

Vectors

Data frame

List

Datatypes / Attributes

} nokkonam.